Milestone 4: Randomized MAC Protocol Simulation

Scribe group 13 Networks

Scribe Question 1: Baseline System and Motivation for Randomization

The Deterministic Baseline (Milestone 3)

the deterministic baseline simulation is done for the Binary Exponential Backoff (BEB) MAC protocol, in which we had 10 nodes that were sharing a channel about 50,000 time slots. Every node has its own queue, Contention Window (CW) and a backoff counter. after each slot the backoff counter decreases when there are packets to transmit and when the backoff reaches zero it transmits..

The key deterministic element is the **backoff selection formula**

$$\textbf{self.backoff} = (CW//2 + node\_id)\%CW$$

This formula is applied thrice. For initialization, after a successful transmission and after a collision. There is no randomness in this algorithm whatsoever and the backoff value depends on the node's ID and the CW.

Why Introduce Randomness?

the deterministic backoff selection formula has a problem, it selects a unique and fixed backoff slot to every node according to its ID. What this does is that it creates a nearly perfect schedule where nodes almost never collide, which is not realistic. In real networks nodes are not globally synchronized, nodes can enter or leave the network dynamically. the formula becomes biased as the nodes with lower IDs get shorter backoff. It does not show the real working of the IEEE 802.11 DCF working.

According to the paper the BEB algorithm needs a backoff time that is uniformly taken from $[0, CW - 1]$. And this controlled randomness helps BEBA with its collision resolution power.

What Part Is Being Modified?

The backoff selection rule that assigns a backoff counter after initialization, success and collision is being replaced by a random one whoch is a random draw.

Scribe Question 2: Randomized Algorithm Design

What Decision Component Now Involves Randomness?

The backoff counter selection formula is randomized and it used at, Node initialization: First setup for nodes, After a successful transmission: CW resets to `CW_MIN` and After a collision: CW doubles up to `CW_MAX`

Probability Distribution Used

A discrete uniform distribution over the integer range $[0, CW - 1]$ is used. Numpy's random number generator is used to sample. All the elements in $[0, CW - 1]$ is equally likely.

How Does It Differ from the Deterministic Baseline?

Aspect	Deterministic	Randomized
Backoff formula	Node-ID based formula	Uniform random draw
Depends on node ID	Yes	No
Same backoff every time	Yes (for same CW)	No, varies each time
Follows IEEE 802.11	No	Yes
Collision resolution	Predictable	Probabilistic

Algorithm Description

1. First based on the p each node is initialized with a fixed arrival interval. The Contention window is set to `CW_MIN` and a backoff value is selected on random with equal probability of each value in that range being picked.
2. In every time slots packets arrive and are queued in the nodes and as the time slot change the queue packet decrements the backoff by 1
3. Nodes with backoff equal to zero attempt to transmit
4. When exactly 1 node has transmitted it is considered a success, the queue decrements, CW is reset to `CW_MIN` and now backoff is selected from $[0, \texttt{CW\_MIN} - 1]$
5. If 2 or more nodes transmit at the same it then it is considered as a collision, then CW doubles up to `CW_MAX` and and now backoff is selected from $[0, CW - 1]$

Scribe Question 3: Probabilistic Reasoning Behind the Algorithm

How Does Randomness Address Uncertainty?

In a common channel, there is no information available to one or more nodes about what other nodes are doing. This is the fundamental uncertainty of the distributed MAC protocols. A deterministic backoff assigns fixed slots but this only works in case every node is coordinated, which is impossible in a decentralized system. This is solved by randomness where backoff of each node is made to be independently and unpredictably chosen. Although when there is a collision of two nodes, the likelihood of choosing the same repeated backoff decreases considerably with the increase of CW:

$$P(\text{two nodes collide again}) = \frac{1}{CW}$$

As CW doubles after each collision, this probability halves each round.

Probabilistic Assumptions Supporting the Design

The research paper uses a discrete time Markov model The reference paper formalizes this with a **discrete-time Markov chain** model. The key assumptions are:

1. At the beginning of the slot the attempt of transmitting which is the **attempt probability** $\tau = \frac{2}{CW}$
2. The channel state (Idle, Success, Collision) can be derived from:

$$P_I = (1-\tau)^N$$

$$P_S = N\tau(1-\tau)^{N-1}$$

$$P_C = 1 - P_I - P_S$$

3. Post-busy slots are not like other slots in that only the nodes that have just issued a transmission can be able to immediately retransmit. This uneven access is one of the basic characteristics of real BEB that the randomized model can represent well.

Why Is This Expected to Improve Performance?

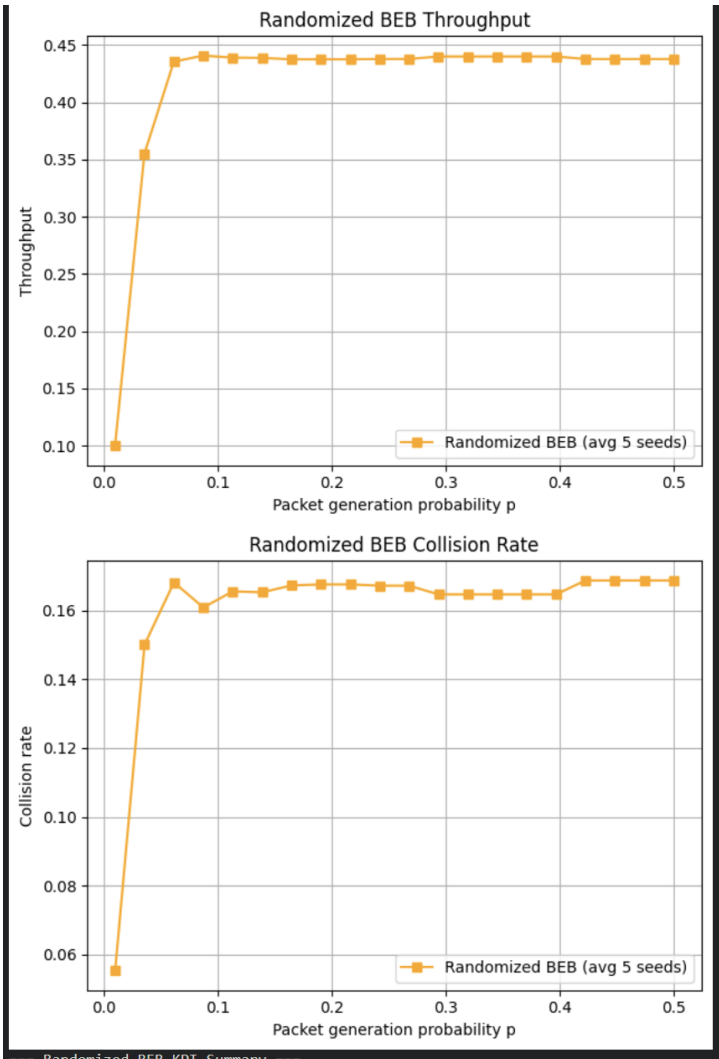
Random backoffs spread over time the attempts of transmission, and the possibility of repeated collisions is reduced. None of the nodes receives backoff which is systematically shorter or longer because all are drawn on the same distribution. The algorithm is independent of the number of nodes and the IDs of nodes.

Scribe Question 4: Implementation within the Simulation Pipeline

Complete Code Implementations

4.1.1 Version 1: Fully Randomized BEB

```python
import numpy as np
import matplotlib.pyplot as plt
N = 10
T = 50000
CW_MIN = 4
CW_MAX = 1024
NUM_SEEDS = 5
p_values = np.linspace(0.01, 0.5, 20)
class Node:

    def __init__(self, p, node_id, rng):
        self.p = p
        self.node_id = node_id
        self.rng = rng

        self.arrival_interval = max(1, int(round(1 / p)))

        self.queue = 0
        self.cw = CW_MIN
        self.backoff = self.rng.integers(0, self.cw)

    def arrival(self, slot):
        if (slot + self.node_id) % self.arrival_interval == 0:
            self.queue += 1

    def decrement_backoff(self):
        if self.queue > 0 and self.backoff > 0:
            self.backoff -= 1

    def ready(self):
        return self.queue > 0 and self.backoff == 0

    def success(self):
        self.queue -= 1
        self.cw = CW_MIN
        self.backoff = self.rng.integers(0, self.cw)

    def collision(self):
        self.cw = min(self.cw * 2, CW_MAX)
        self.backoff = self.rng.integers(0, self.cw)

def run_simulation(p, seed):
    rng = np.random.default_rng(seed)
    nodes = [Node(p, i, rng) for i in range(N)]
    successes = 0
    collisions = 0
    for slot in range(T):

        for node in nodes:
            node.arrival(slot)

        for node in nodes:
            node.decrement_backoff()

        transmitters = [node for node in nodes if node.ready()]

        if len(transmitters) == 1:
            successes += 1
            transmitters[0].success()

        elif len(transmitters) > 1:
            collisions += 1
            for node in transmitters:
                node.collision()

    throughput = successes / T
    collision_rate = collisions / T

    return throughput, collision_rate

rand_throughputs = []
rand_collisions = []
for p in p_values:
    thr_runs = []
    col_runs = []

    for seed in range(NUM_SEEDS):
        thr, col = run_simulation(p, seed)
        thr_runs.append(thr)
        col_runs.append(col)

    rand_throughputs.append(np.mean(thr_runs))
    rand_collisions.append(np.mean(col_runs))

plt.figure()
plt.plot(p_values, rand_throughputs, label=f"Randomized BEB (avg {NUM_SEEDS} seeds)",
         linestyle='-', marker='s', color='orange')
plt.xlabel("Packet generation probability p")
plt.ylabel("Throughput")
plt.title("Randomized BEB Throughput")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

plt.figure()
plt.plot(p_values, rand_collisions, label=f"Randomized BEB (avg {NUM_SEEDS} seeds)",
         linestyle='-', marker='s', color='orange')
plt.xlabel("Packet generation probability p")
plt.ylabel("Collision rate")
plt.title("Randomized BEB Collision Rate")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
print("== Randomized BEB KPI Summary ==")
print(f"{'p':>6} | {'Throughput':>12} | {'Collision Rate':>14}")
print("-" * 38)
for i, p in enumerate(p_values):
    print(f"{p:>6.2f} | {rand_throughputs[i]:>12.4f} | {rand_collisions[i]:>14.4f}")
```

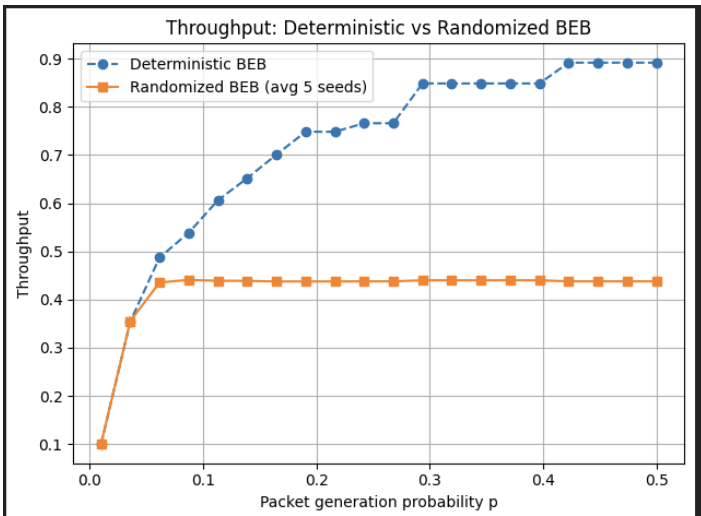


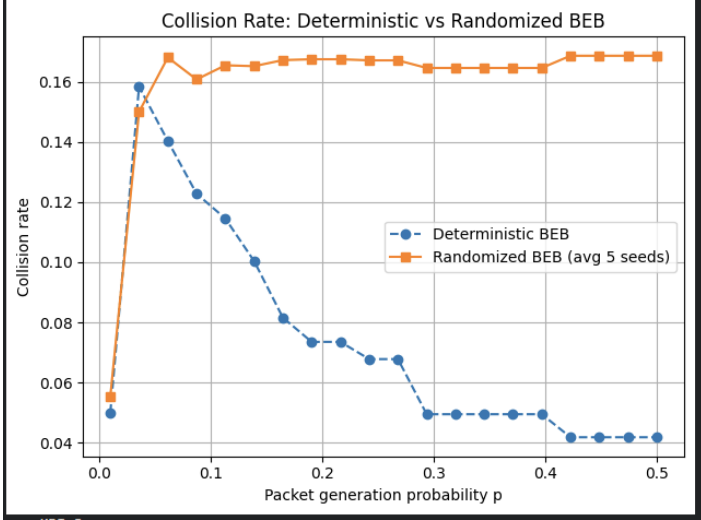

```
=== Randomized BEB KPI Summary ===
     p |    Throughput | Collision Rate
-------------------------------------
  0.01 |        0.0999 |         0.0553
  0.04 |        0.3551 |         0.1501
  0.06 |        0.4355 |         0.1682
  0.09 |        0.4407 |         0.1609
  0.11 |        0.4390 |         0.1656
  0.14 |        0.4387 |         0.1653
  0.16 |        0.4375 |         0.1673
  0.19 |        0.4375 |         0.1676
  0.22 |        0.4375 |         0.1676
  0.24 |        0.4378 |         0.1672
  0.27 |        0.4378 |         0.1672
  0.29 |        0.4399 |         0.1647
  0.32 |        0.4399 |         0.1647
  0.35 |        0.4399 |         0.1647
  0.37 |        0.4399 |         0.1647
  0.40 |        0.4399 |         0.1647
  0.42 |        0.4377 |         0.1687
  0.45 |        0.4377 |         0.1687
  0.47 |        0.4377 |         0.1687
  0.50 |        0.4377 |         0.1687
```

4.1.2 Version 2: Deterministic vs Randomized BEB (Comparison)

```python
import numpy as np
import matplotlib.pyplot as plt
N = 10
T = 50000
CW_MIN = 4
CW_MAX = 1024
NUM_SEEDS = 5
p_values = np.linspace(0.01, 0.5, 20)
class Node:

    def __init__(self, p, node_id, rng):
        self.p = p
        self.node_id = node_id
        self.rng = rng

        self.arrival_interval = max(1, int(round(1 / p)))

        self.queue = 0
        self.cw = CW_MIN

        self.backoff = self.rng.integers(0, self.cw)

    def arrival(self, slot):
        if (slot + self.node_id) % self.arrival_interval == 0:
            self.queue += 1

    def decrement_backoff(self):
        if self.queue > 0 and self.backoff > 0:
            self.backoff -= 1

    def ready(self):
        return self.queue > 0 and self.backoff == 0

    def success(self):
        self.queue -= 1
        self.cw = CW_MIN
        # Random backoff from [0, CW_MIN - 1]
        self.backoff = self.rng.integers(0, self.cw)

    def collision(self):
        self.cw = min(self.cw * 2, CW_MAX)
        # Random backoff from [0, new_CW - 1]
        self.backoff = self.rng.integers(0, self.cw)


def run_simulation(p, seed):

    rng = np.random.default_rng(seed)
    nodes = [Node(p, i, rng) for i in range(N)]

    successes = 0
    collisions = 0

    for slot in range(T):

        for node in nodes:
            node.arrival(slot)

        for node in nodes:
            node.decrement_backoff()

        transmitters = [node for node in nodes if node.ready()]

        if len(transmitters) == 1:
            successes += 1
            transmitters[0].success()

        elif len(transmitters) > 1:
            collisions += 1
            for node in transmitters:
                node.collision()
```

```
    throughput = successes / T
    collision_rate = collisions / T
    return throughput, collision_rate

rand_throughputs = []
rand_collisions = []

for p in p_values:
    thr_runs = []
    col_runs = []

    for seed in range(NUM_SEEDS):
        thr, col = run_simulation(p, seed)
        thr_runs.append(thr)
        col_runs.append(col)

    rand_throughputs.append(np.mean(thr_runs))
    rand_collisions.append(np.mean(col_runs))

class DetNode:
    def __init__(self, p, node_id):
        self.p = p
        self.node_id = node_id
        self.arrival_interval = max(1, int(round(1 / p)))
        self.queue = 0
        self.cw = CW_MIN
        self.backoff = (CW_MIN // 2 + node_id) % CW_MIN

    def arrival(self, slot):
        if (slot + self.node_id) % self.arrival_interval == 0:
            self.queue += 1

    def decrement_backoff(self):
        if self.queue > 0 and self.backoff > 0:
            self.backoff -= 1

    def ready(self):
        return self.queue > 0 and self.backoff == 0

    def success(self):
        self.queue -= 1
        self.cw = CW_MIN
        self.backoff = (self.cw // 2 + self.node_id) % self.cw

    def collision(self):
        self.cw = min(self.cw * 2, CW_MAX)
        self.backoff = (self.cw // 2 + self.node_id) % self.cw

def run_det_simulation(p):
    nodes = [DetNode(p, i) for i in range(N)]
    successes = 0
    collisions = 0
    for slot in range(T):
        for node in nodes:
            node.arrival(slot)
        for node in nodes:
            node.decrement_backoff()
        transmitters = [node for node in nodes if node.ready()]
        if len(transmitters) == 1:
            successes += 1
            transmitters[0].success()
        elif len(transmitters) > 1:
            collisions += 1
            for node in transmitters:
                node.collision()
    return successes / T, collisions / T

det_throughputs = []
det_collisions = []
for p in p_values:
    thr, col = run_det_simulation(p)
    det_throughputs.append(thr)
    det_collisions.append(col)

plt.figure()
plt.plot(p_values, det_throughputs, label="Deterministic BEB", linestyle='--', marker='o')
plt.plot(p_values, rand_throughputs, label=f"Randomized BEB (avg {NUM_SEEDS} seeds)", linestyle='-', marker='s')
plt.xlabel("Packet generation probability p")
plt.ylabel("Throughput")
plt.title("Throughput: Deterministic vs Randomized BEB")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

plt.figure()
plt.plot(p_values, det_collisions, label="Deterministic BEB", linestyle='--', marker='o')
plt.plot(p_values, rand_collisions, label=f"Randomized BEB (avg {NUM_SEEDS} seeds)", linestyle='-', marker='s')
plt.xlabel("Packet generation probability p")
plt.ylabel("Collision rate")
plt.title("Collision Rate: Deterministic vs Randomized BEB")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print("=== KPI Summary ===")
print(f"{'p':>6} | {'Det Thr':>10} | {'Rand Thr':>10} | {'Det Col':>10} | {'Rand Col':>10}")
print("-" * 56)
for i, p in enumerate(p_values):
    print(f"{p:>6.2f} | {det_throughputs[i]:>10.4f} | {rand_throughputs[i]:>10.4f} | "
          f"{det_collisions[i]:>10.4f} | {rand_collisions[i]:>10.4f}")
```

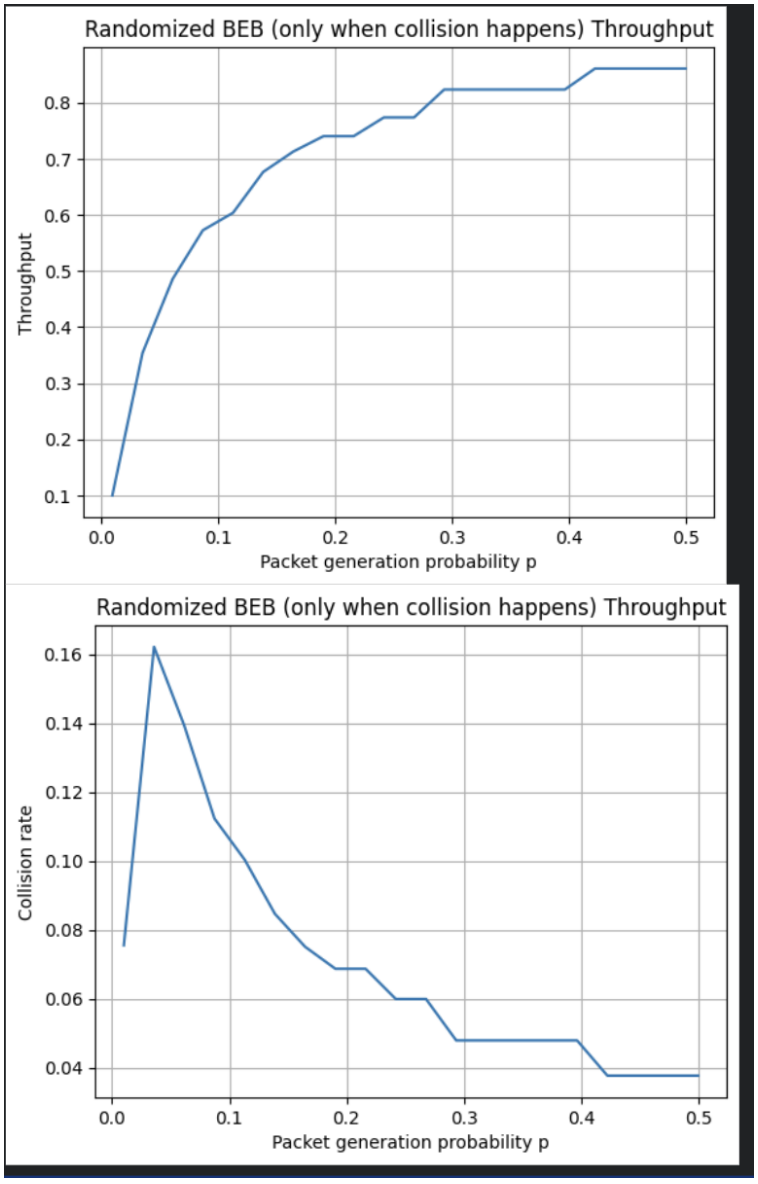





```
=== KPI Summary ===
     p |    Det Thr |   Rand Thr |    Det Col |   Rand Col
-------------------------------------------------------------
  0.01 |     0.0999 |     0.0999 |     0.0500 |     0.0553
  0.04 |     0.3541 |     0.3551 |     0.1585 |     0.1501
  0.06 |     0.4872 |     0.4355 |     0.1404 |     0.1682
  0.09 |     0.5386 |     0.4407 |     0.1228 |     0.1609
  0.11 |     0.6061 |     0.4390 |     0.1146 |     0.1656
  0.14 |     0.6518 |     0.4387 |     0.1004 |     0.1653
  0.16 |     0.7007 |     0.4375 |     0.0816 |     0.1673
  0.19 |     0.7482 |     0.4375 |     0.0735 |     0.1676
  0.22 |     0.7482 |     0.4375 |     0.0735 |     0.1676
  0.24 |     0.7659 |     0.4378 |     0.0678 |     0.1672
  0.27 |     0.7659 |     0.4378 |     0.0678 |     0.1672
  0.29 |     0.8485 |     0.4399 |     0.0494 |     0.1647
  0.32 |     0.8485 |     0.4399 |     0.0494 |     0.1647
  0.35 |     0.8485 |     0.4399 |     0.0494 |     0.1647
  0.37 |     0.8485 |     0.4399 |     0.0494 |     0.1647
  0.40 |     0.8485 |     0.4399 |     0.0494 |     0.1647
  0.42 |     0.8916 |     0.4377 |     0.0418 |     0.1687
  0.45 |     0.8916 |     0.4377 |     0.0418 |     0.1687
  0.47 |     0.8916 |     0.4377 |     0.0418 |     0.1687
  0.50 |     0.8916 |     0.4377 |     0.0418 |     0.1687
```

4.1.3 Version 3: Hybrid BEB (Randomized Only on Collision)

```python
import numpy as np
import matplotlib.pyplot as plt
N = 10
T = 50000
CW_MIN = 4
CW_MAX = 1024
NUM_RUNS = 5
p_values = np.linspace(0.01,0.5,20)
class Node:

    def __init__(self,p,node_id):

        self.p = p
        self.node_id = node_id

        # Same deterministic arrivals
        self.arrival_interval = max(1,int(round(1/p)))

        self.queue = 0
        self.cw = CW_MIN

        # SAME deterministic start
        self.backoff = (CW_MIN//2 + node_id) % CW_MIN

    def arrival(self,slot):

        if (slot + self.node_id) % self.arrival_interval == 0:
            self.queue += 1

    def decrement_backoff(self):

        if self.queue > 0 and self.backoff > 0:
            self.backoff -= 1

    def ready(self):

        return self.queue > 0 and self.backoff == 0

    def success(self):

        self.queue -= 1

        # reset CW
        self.cw = CW_MIN

        # KEEP deterministic reset
        self.backoff = (self.cw//2 + self.node_id) % self.cw

    def collision(self):

        # BEB growth
        self.cw = min(self.cw*2,CW_MAX)

        # RANDOM ONLY HERE
        self.backoff = np.random.randint(0,self.cw)

def run_simulation(p,seed):

    np.random.seed(seed)

    nodes = [Node(p,i) for i in range(N)]

    successes = 0
    collisions = 0

    for slot in range(T):

        for node in nodes:
            node.arrival(slot)

        for node in nodes:
            node.decrement_backoff()

        transmitters = [node for node in nodes if node.ready()]

        if len(transmitters) == 1:

            successes += 1
            transmitters[0].success()

        elif len(transmitters) > 1:

            collisions += 1

            for node in transmitters:
                node.collision()

    return successes/T, collisions/T

throughputs = []
collisions = []

for p in p_values:

    thr_runs = []
    col_runs = []

    for run in range(NUM_RUNS):

        thr,col = run_simulation(p,seed=run)

        thr_runs.append(thr)
        col_runs.append(col)

    throughputs.append(np.mean(thr_runs))
    collisions.append(np.mean(col_runs))
plt.figure()
plt.plot(p_values,throughputs)
plt.xlabel("Packet generation probability p")
plt.ylabel("Throughput")
plt.title("Randomized BEB (only when collision happens) Throughput")
plt.grid()
plt.show()
plt.figure()
plt.plot(p_values,collisions)
plt.xlabel("Packet generation probability p")
plt.ylabel("Collision rate")
plt.title("Randomized BEB (only when collision happens) Throughput")
plt.grid()
plt.show()
```


Randomized BEB (only when collision happens) Throughput
Throughput
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.0
0.1
0.2
0.3
0.4
0.5
Packet generation probability p
Randomized BEB (only when collision happens) Throughput
Collision rate
0.04
0.06
0.08
0.10
0.12
0.14
0.16
0.0
0.1
0.2
0.3
0.4
0.5
Packet generation probability p

What Changes Were Made?

Three versions of the algorithm were implemented

Version 1: Fully Randomized BEB:

The backoff selection formula is replaced with a uniform random one at all three points that is initialization, after success, and after collision:

- At initialization: backoff $\sim$ Uniform$[0, \texttt{CW_MIN} - 1]$
- After success: CW resets to `CW_MIN`, backoff $\sim$ Uniform$[0, \texttt{CW_MIN} - 1]$
- After collision: CW doubles, backoff $\sim$ Uniform$[0, \text{new_CW} - 1]$

Nothing is deterministic and the backoff selection is independent ans uniformly sampled.

Version 2: Deterministic vs Randomized Comparison:

In this version, the deterministic baseline algorithm is run side by side with the fully randomized algorithm. The node ID based method is used in the deterministic baseline and the uniform random draws are used in the randomized version, and formula at all three backoff points. The two are plotted concurrently because they need to be directly compared using throughput and collision rate across all values of p.

Version 3: Hybrid BEB (Randomized Only on Collision):

Randomness is introduced only if collision occurs and the initialization and success backoff remains deterministic:

- At initialization (deterministic): backoff = $(\texttt{CW\_MIN}/2 + \text{node\_id})\%\texttt{CW\_MIN}$
- After success (deterministic): backoff = $(\texttt{CW}/2 + \text{node\_id})\%\texttt{CW}$
- After collision (randomized): backoff $\sim$ Uniform$[0, \text{new_CW} - 1]$

How Is Randomness Generated?

Versions 1 and 2 use NumPy's modern `default_rng(seed)` generator. Version 3 uses NumPy's legacy `np.random.seed(seed)`

Scribe Question 5: Evaluation Metrics and Comparative Results

Evaluation Metrics

Two KPIs are used:

1. Throughput $= \frac{\text{successful transmissions}}{\text{total slots}}$

Measures how efficiently the channel is used

2. Collision Rate $= \frac{\text{collisions}}{\text{total slots}}$

Measures how often two or more nodes transmit simultaneously

Both metrics are averaged across 5 random seeds to reduce variance.

Comparative Results and Interpretation

Throughput Comparison:

p range	Deterministic	Randomized
Explanation		
Low (0.01–0.05)	Rising steeply	Rising slowly
Medium (0.1–0.3)	0.6–0.85	∼0.44 (flat)
High (0.3–0.5)	∼0.90	∼0.44 (flat)

Table 1: Throughput comparison across p values

Collision Rate Comparison:

p range	Deterministic	Randomized
Low (0.01–0.05)	∼0.05–0.16	∼0.05–0.17
Medium–High	Drops to ∼0.04	Stays at ∼0.165

Interpretation

The deterministic algorithm has extremely high throughput (∼0.90), however, this occurs due to the deterministic backoff pattern which accidentally forms a repeating transmission order. This causes the result to appear better than it is and is not realistic network behavior.

This randomized algorithm levels off at about ∼0.44 throughput and a constant collision rate of ∼0.165. This is how a real Binary Exponential Backoff (BEB) protocol is supposed to work. Thus, the randomized algorithm is much more realistic and reliable.

Scribe Question 6: Insights and Remaining Challenges

Insights Gained

The experiments demonstrated deterministic backoff as not a very robust method, since it is based on fixed node identities and controlled conditions, which are not present in the real network. With the addition of randomness, more realistic contention behavior is generated; an uninterrupted collision rate is a sign of normal system operation and not failure. It was also found that one should average across several random seeds in order to derive consistent and stable KPIs, as individual runs can be very different. Also, post-transmission slot behaviour of the system is also of significance and this is automatically taken care of by randomized backoff, as opposed to deterministic approaches. Lastly, BEB protocol performance is very sensitive to the selection of the backoff range and in this case, any minor shift in the parameters may have substantial effect on throughput.

When Does Randomized Perform Better or Worse?

The randomized strategy works better than the deterministic baseline in the medium to high traffic load, where many nodes try to transmit at once. In this situation, the randomness helps to minimize synchronized collisions and leads to a better system stability.

However, the randomized strategy could be similar or slightly inferior to the deterministic approach under very low traffic loads where collisions are already rare and the benefit of randomization is minimal.

Remaining Challenges

The results show that the throughput limit of about $\sim$0.44 at high packet probabilities could potentially be improved with the help of more complex techniques like adaptive contention window tuning based on the load in the network. The steady collision rate of about $\sim$0.165 further suggests smarter backoff strategies such as load-aware or non-uniform random selection could further improve performance. In addition, even though the averaging over multiple seeds improves the stability, the number of seeds and the simulation length will still affect the results, and it means that the confidence interval analysis will work more reliably when evaluating.